



Detección de Fraude en Transacciones Bancarias mediante Técnicas Clásicas de Machine Learning



UNIVERSITAT ROVIRA I VIRGILI

Kevin Alexander Roman Zaruma
Emanuel Nestor Escobar Juipa

Artificial Intelligence Foundations

Abstract

El presente proyecto desarrolló un sistema de detección de fraude en transacciones de tarjetas de crédito mediante técnicas clásicas de *Machine Learning* (ML). Se utilizó un dataset público compuesto por 284,807 transacciones y 31 variables, donde únicamente el 0.17% correspondía a operaciones fraudulentas, evidenciando un fuerte desbalance de clases. El trabajo incluyó exploración y visualización de datos, preprocesamiento, escalado de variables y entrenamiento de modelos de clasificación binaria y una optimización de hiperparámetros. Se implementaron *Logistic Regression* y *Random Forest* utilizando estrategias de balanceo mediante *class_weight='balanced'*. *Logistic Regression* obtuvo un recall de 91.84%, detectando 90 de 98 fraudes, aunque con baja precisión (6.09%) debido al elevado número de falsos positivos. Por su parte, *Random Forest* alcanzó un accuracy de 99.95% y una precisión de 87.10%, mostrando un desempeño más equilibrado. Finalmente, se realizó optimización de hiperparámetros mejorando la precisión a 89.10%, el modelo optimizado mostró una alta capacidad de discriminación entre transacciones fraudulentas y legítimas.

ÍNDICE

Abstract	1
1. Introducción	3
2. Formulación del Problema	4
3. Configuración del entorno de desarrollo	4
3.1 Creación del entorno virtual.....	5
3.2 Instalación de librerías	5
4. Carga y exploración del dataset.....	5
4.1 Carga del dataset.....	5
4.2 Exploración inicial de datos	6
5. Distribución de clases	7
5.1 Observación importante	7
6. Visualización de datos	8
7. Preprocesamiento de datos	11
7.1 División del dataset.....	11
7.2 Escalado de variables.....	11
7.3. Verificar balance de clases	11
8. Entrenamiento del modelo.....	12
9. Evaluación del modelo	13
9.1 Resultados obtenidos	13
9.2 Interpretación de resultados.....	14
9.3 Matriz de confusión	15
10. Comparación de modelos.....	16
10.1 Resultados obtenidos.....	16
10.2 Interpretación	16
11. Ajuste de Hiperparámetros	18
11.1 Definición de parámetros	18
12. Conclusiones	20

1. Introducción

El fraude financiero, en los últimos 10 años, representa uno de los principales desafíos para las instituciones bancarias modernas debido a las grandes pérdidas económicas y riesgos de seguridad asociados. El incremento de las transacciones digitales, el comercio electrónico y el uso exponencial de plataformas bancarias en línea ha favorecido la aparición de nuevas modalidades de fraude cada vez más sofisticadas y difíciles de detectar.

Las técnicas de ML permiten detectar patrones complejos y anomalías en grandes volúmenes de datos bancarios de manera automática, identificando comportamientos inusuales que podrían pasar desapercibidos mediante métodos tradicionales. Debido a su capacidad de aprendizaje de datos, estos modelos pueden identificar relaciones no lineales, tendencias ocultas y cambios de comportamiento de los datos.

El objetivo de este proyecto fue desarrollar un sistema de detección de fraude en transacciones bancarias utilizando técnicas de *Machine Learning*. Para ello, se trabajó con un dataset de transacciones de tarjetas de crédito y se entrenó distintos modelos capaces de identificar operaciones fraudulentas.

En esta primera fase del proyecto se ha realizado la preparación del entorno de desarrollo, la carga y exploración inicial del dataset, el preprocesamiento del dataset y el entrenamiento de modelos de clasificación utilizando Logistic Regression y Random Forest y finalmente, se aplicó la optimización de hiperparámetros.

2. Formulación del Problema

Tipo de aprendizaje: Supervisado

Tipo de tarea: Clasificación binaria

Objetivo: Detectar automáticamente transacciones fraudulentas de la base de datos de Detección de fraude de Tarjetas de Crédito de Kaggle.

Variable objetivo:

El problema corresponde a una tarea de clasificación binaria, donde el modelo debe asignar cada transacción a una de dos clases posibles: fraudulenta o legítima.

Clases (binario)

0 = legítima

1 = fraudulenta

Métrica principal:

En problemas de detección de fraude en transacciones, el recall resulta especialmente relevante debido a que minimizar los fraudes no detectados constituye una prioridad operacional.

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

3. Configuración del entorno de desarrollo

Para el desarrollo del proyecto se ha utilizado:

- Python
- Visual Studio Code
- OpenCode GitHub como asistente de programación
- Entorno virtual mediante venv

3.1 Creación del entorno virtual

Se creó un entorno virtual para aislar las dependencias del proyecto y evitar conflictos con otras instalaciones de Python.

Comando utilizado:

```
python -m venv venv
```

3.2 Instalación de librerías

Se instalaron las librerías necesarias para el desarrollo del proyecto:

```
pip install pandas, numpy, scikit-learn, matplotlib, seaborn  
streamlit, joblib
```

Las principales librerías utilizadas fueron:

Librería	Función principal
pandas	Manipulación de datos
numpy	Operaciones numéricas
scikit-learn	Machine Learning
matplotlib	Visualización
seaborn	Gráficos estadísticos
streamlit	Interfaz web
joblib	Guardado de modelos

4. Carga y exploración del dataset

Se utilizó el dataset de detección de fraude en tarjetas de crédito, ampliamente empleado en proyectos de Machine Learning relacionados con clasificación binaria.

4.1 Carga del dataset

Dataset obtenido desde Kaggle: “Credit Card Fraud Detection Dataset”.

Código utilizado:

```
import pandas as pd
```

```
df = pd.read_csv("creditcard.csv")
```

4.2 Exploración inicial de datos

Se visualizaron las primeras filas del dataset para comprobar la estructura de las variables y validar que la carga se había realizado correctamente.

Resultados obtenidos

El dataset contiene:

- 284.807 registros
- 31 columnas

Entre las variables principales se encuentran:

- Time
- Amount
- Las variables V1–V28 corresponden a componentes principales obtenidos mediante PCA para preservar la confidencialidad de los datos originales.
- Class (variable objetivo)

La columna **Class** representa:

Valor	Significado
0	Transacción Legítima
1	Transacción Fraudulenta

Figura 1. Visualización de variables del dataset

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22	V23	V24	V25	V26	V27	V28	Amount	Class
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	-0.189115	0.133558	-0.021053	149.62	0
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	0.125895	-0.008983	0.014724	2.69	0
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	-0.139097	-0.055353	-0.059752	378.66	0
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	-0.221929	0.062723	0.061458	123.50	0
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	0.502292	0.219422	0.215153	69.99	0

5 rows × 31 columns

En la Figura 2, se puede visualizar el tipo de variables:

- float 64 (time, V1 – V28 y amount)
- un int64 (Class)

Figura 2. Tipos de variables del dataset

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
#   Column      Non-Null Count  Dtype  
---  -
0   Time        284807 non-null float64
1   V1          284807 non-null float64
2   V2          284807 non-null float64
3   V3          284807 non-null float64
4   V4          284807 non-null float64
5   V5          284807 non-null float64
6   V6          284807 non-null float64
7   V7          284807 non-null float64
8   V8          284807 non-null float64
9   V9          284807 non-null float64
10  V10         284807 non-null float64
11  V11         284807 non-null float64
12  V12         284807 non-null float64
13  V13         284807 non-null float64
14  V14         284807 non-null float64
15  V15         284807 non-null float64
16  V16         284807 non-null float64
17  V17         284807 non-null float64
18  V18         284807 non-null float64
19  V19         284807 non-null float64
...
29  Amount      284807 non-null float64
30  Class       284807 non-null int64  
dtypes: float64(30), int64(1)
memory usage: 67.4 MB
```

5. Distribución de clases

Se analizó la distribución entre transacciones fraudulentas y no fraudulentas.

Resultados

Clase	Cantidad
Legítima (0)	284.315
Fraudulenta (1)	492

Las transacciones fraudulentas representan únicamente el 0.17% del dataset total.

5.1 Observación importante

Se detectó un fuerte desbalanceo de clases, ya que las transacciones fraudulentas representan una proporción muy pequeña del total del dataset (ver figura 3).

Este aspecto es especialmente importante en Machine Learning, ya que puede provocar que algunos modelos obtengan métricas aparentemente buenas sin detectar correctamente los fraudes reales.

6. Visualización de datos

Se realizó una visualización de la distribución de clases utilizando las librerías seaborn y matplotlib.

Código utilizado:

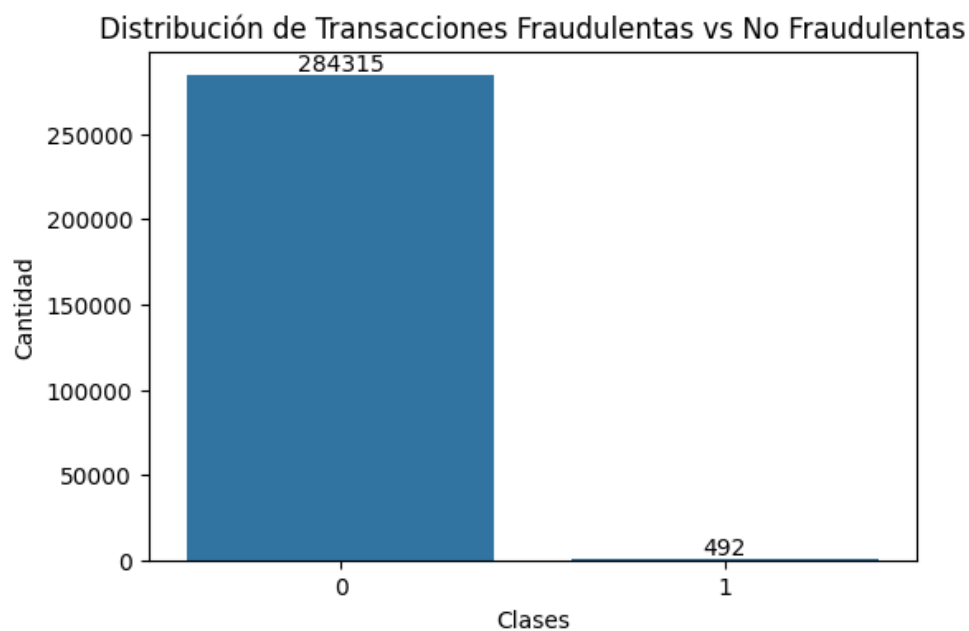
```
sns.countplot(x='Class', data=df)

plt.title("Fraud vs Non-Fraud Transactions")

plt.show()
```

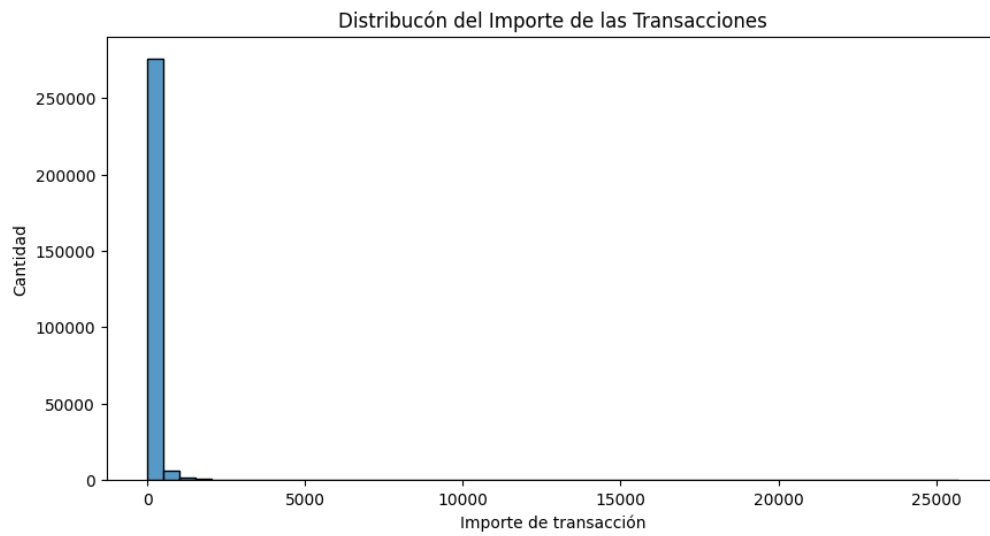
La gráfica permitió observar visualmente el fuerte desbalanceo existente entre ambas clases.

Figura 3. Distribución de Transacciones



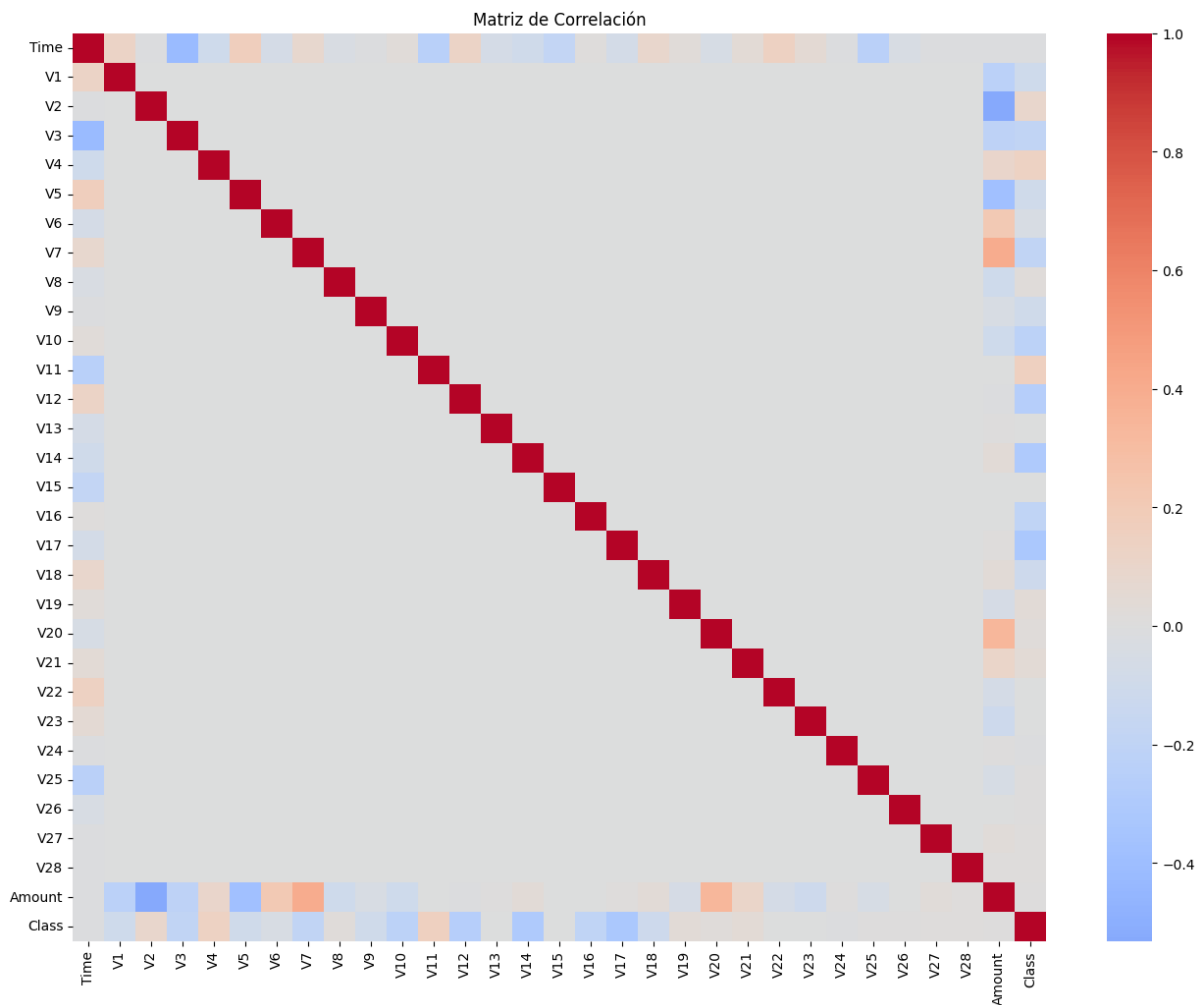
Además, la mayoría de las transacciones son importes pequeños y pocos valores son extremadamente grandes (ver figura 4).

Figura 4. Distribución de importes de transacciones bancarias.



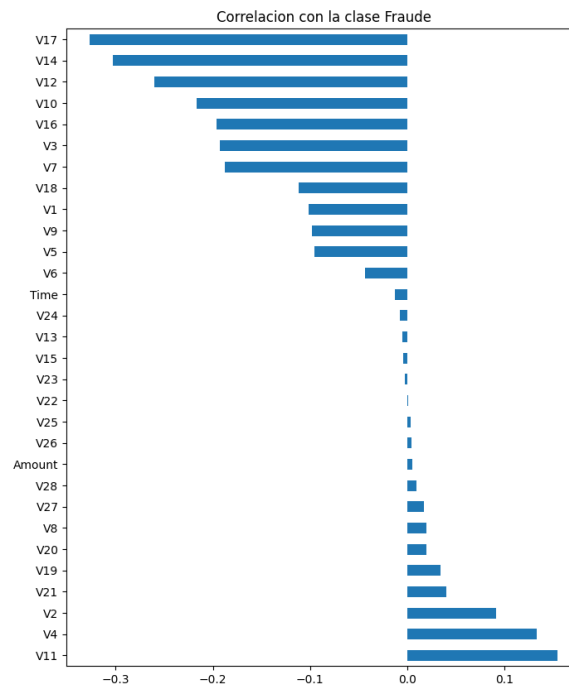
Algunas variables desde el V1 al V18 tiene mayor correlación con la variable Class (ver figura 5).

Figura 5. Matriz de correlación entre todas las variables



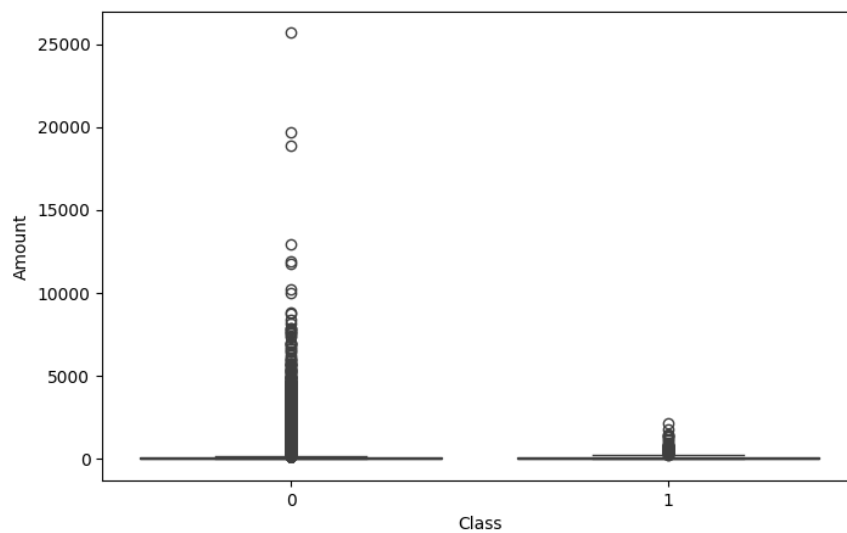
Las variables con correlación positiva aumentan la probabilidad de fraude y las variables negativas disminuyen esa probabilidad (ver figura 6).

Figura 6. Correlación con la clase "Fraude"



Los fraudes tienden a ser en importes más pequeños (ver figura 7).

Figura 7. Importe de la transacción por clase



7. Preprocesamiento de datos

Antes de entrenar el modelo, se realizó un proceso de preprocesamiento de datos.

7.1 División del dataset

Posteriormente, el dataset fue dividido por `train_test_split`

- `test_size=0.2`
80% entrenamiento
20% evaluación

7.2 Escalado de variables

Las variables `Time` y `Amount` fueron normalizadas utilizando `StandardScaler`.

El escalado permitió normalizar las variables numéricas y evitar sesgos derivados de diferencias de magnitud entre atributos.

Código utilizado:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X['Amount'] = scaler.fit_transform(X[['Amount']])

X['Time'] = scaler.fit_transform(X[['Time']])
```

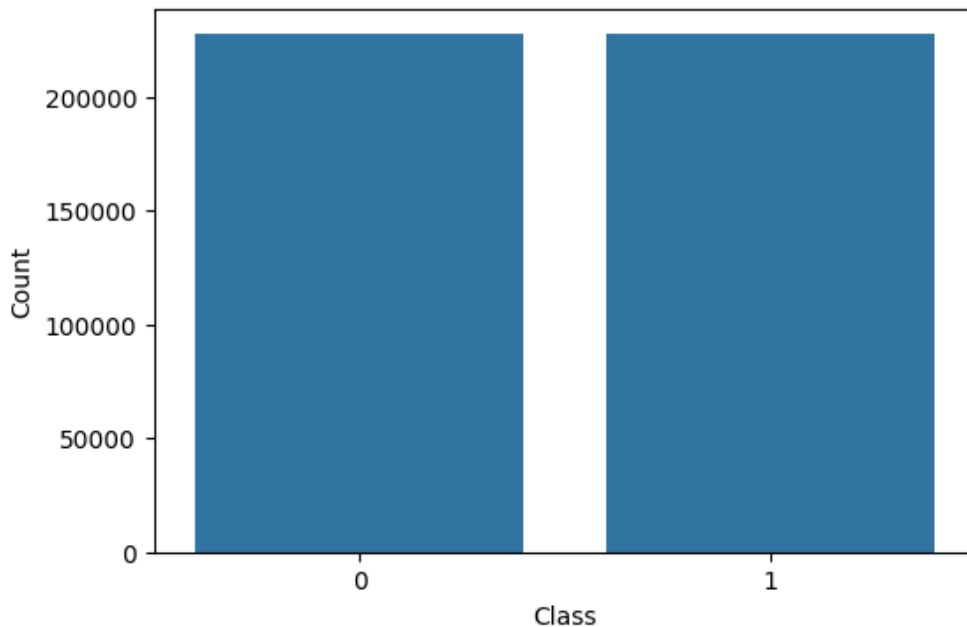
El scaler se ajustó únicamente sobre los datos de entrenamiento para evitar data leakage.

7.3. Verificar balance de clases

El fuerte desbalance de clases puede afectar negativamente el aprendizaje del modelo, favoreciendo excesivamente la clase mayoritaria (legítima). Por este motivo el uso de SMOTE (Synthetic Minority Oversampling Technique) que balancea el dataset (ver figura 8).

El SMOTE generó muestras sintéticas de la clase minoritaria utilizando vecinos cercanos, permitiendo balancear el conjunto de entrenamiento y mejorar la capacidad predictiva del modelo.

Figura 8. Balance de clases después de usar SMOTE



8. Entrenamiento del modelo

Se entrenó un modelo de clasificación utilizando Logistic Regression and Random Forest.

Logistic Regression: fue seleccionada como modelo base debido a su simplicidad, interpretabilidad y eficiencia en problemas de clasificación binaria.

```
class_weight='balanced'
```

```
max_iter=1000
```

Random Forest: fue seleccionado por su capacidad para manejar relaciones no lineales y reducir *overfitting* mediante ensamblaje de árboles.

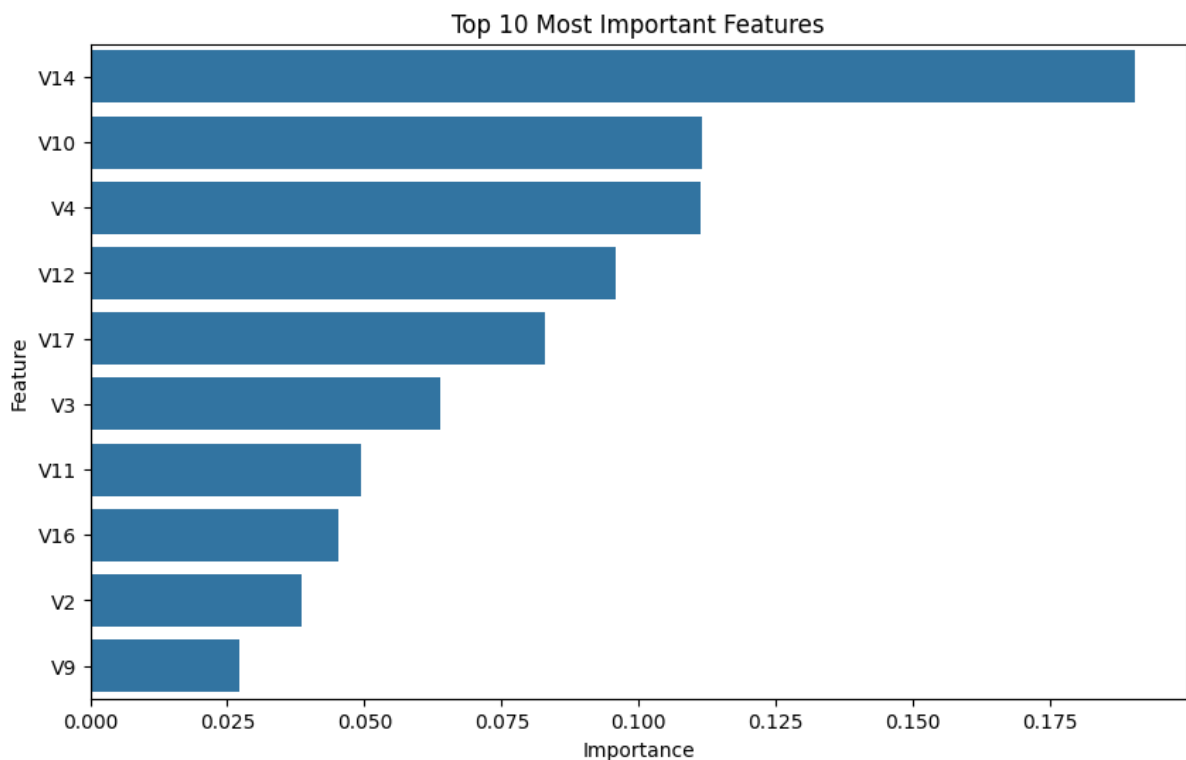
```
n_estimators=100
```

```
class_weight='balanced'
```

`n_estimator`, `class_weight` y `max_itter` ayudan al modelo a dar más importancia a la clase minoritaria y a mejorar la detección de fraudes.

Random Forest (RF) identificó las variables más relevantes para la detección de fraude, permitiendo comprender cuáles componentes tienen mayor influencia en la clasificación.

Además, RF identificó las variables más relevantes (V14, V10, V4) para la detección de fraude, permitiendo comprender cuáles componentes tienen mayor influencia en la clasificación.



9. Evaluación del modelo

El modelo fue evaluado utilizando distintas métricas:

- Accuracy
- Precision
- Recall
- F1-Score
- ROC-AUC

9.1 Resultados obtenidos

Métricas para Logistic Regression (LR):

```
==== Logistic Regression ====  
Accuracy : 0.9741  
Precision: 0.0578  
Recall   : 0.9184  
F1-Score : 0.1088  
ROC-AUC  : 0.9708
```

Métricas para RF

```
==== Random Forest ====  
Accuracy : 0.9995  
Precision: 0.8710  
Recall   : 0.8265  
F1-Score : 0.8482  
ROC-AUC  : 0.9685
```

9.2 Interpretación de resultados

Logistic Regression (LR):

El modelo LR alcanzó un accuracy de 97.41%, mostrando una alta capacidad de clasificación general, lo cual representa un resultado positivo para una primera aproximación.

Sin embargo, la precisión obtenida fue baja, indicando la presencia de una cantidad considerable de falsos positivos.

Esto significa que el modelo marca muchas transacciones legítimas como fraudulentas.

Random Forest (RF):

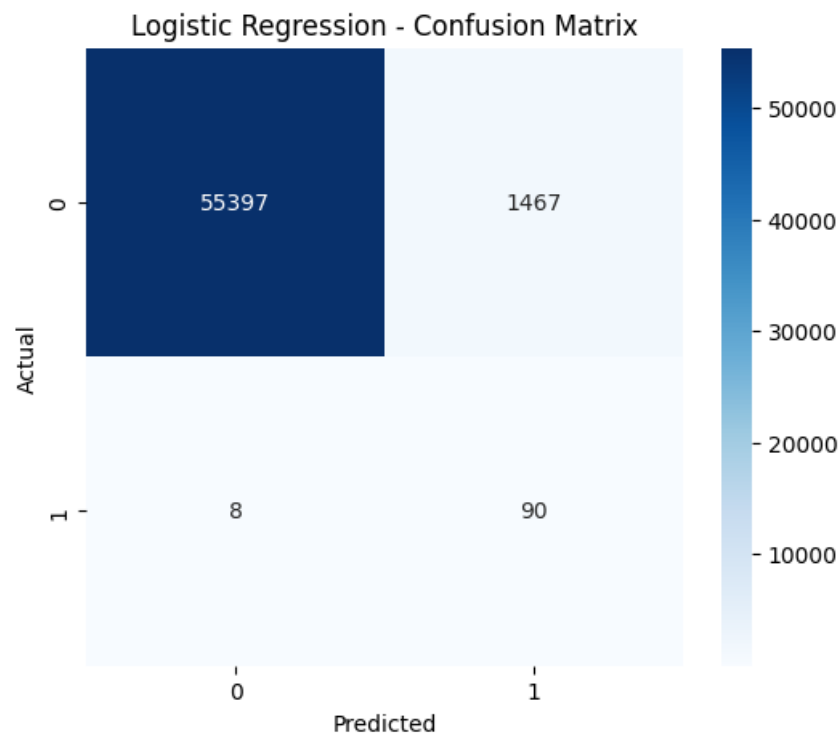
RF alcanzó un accuracy de 99.95%, mostrando un comportamiento altamente estable para la clasificación de transacciones.

Además, la precisión obtenida fue alta, indicando la baja presencia de falsos positivos.

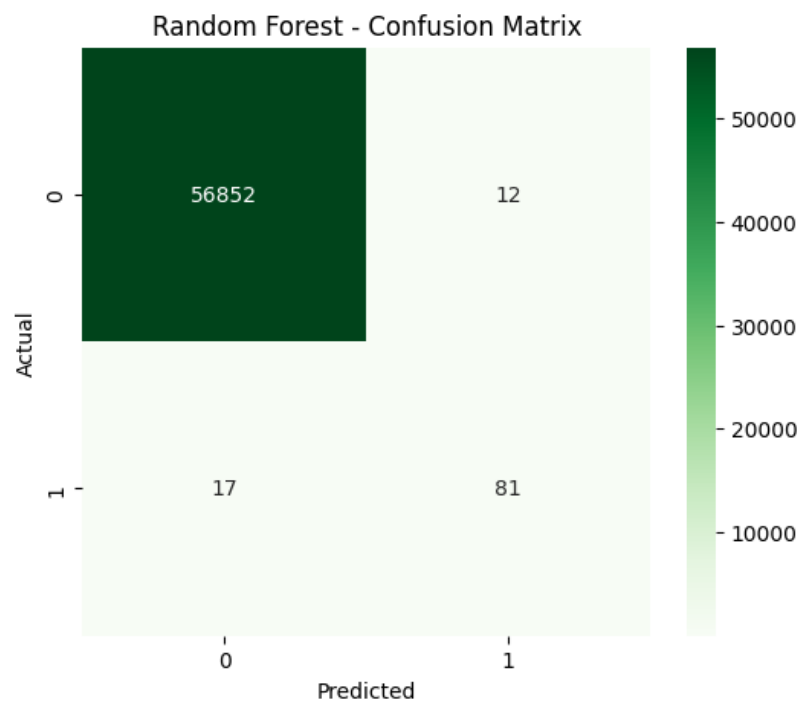
Esto significa que el modelo marca pocas transacciones legítimas como fraudulentas.

9.3 Matriz de confusión

Resultados obtenidos para LR:



Resultados obtenidos para RF:



10. Comparación de modelos

Debido al fuerte desbalanceo del dataset, ambos modelos utilizaron el parámetro para mejorar la detección de transacciones fraudulentas.

```
class_weight='balanced'
```

10.1 Resultados obtenidos

Modelo	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	97.55%	6.09%	91.84%	10.88%	97.08%
Random Forest	99.95%	96.05%	74.49%	84.82%	96.85%

10.2 Interpretación

El modelo LR consiguió detectar una mayor cantidad de fraudes gracias a su alto recall, aunque generó muchos falsos positivos.

Por otro lado, *RF* obtuvo una precisión mucho mayor y redujo significativamente los falsos positivos, ofreciendo un comportamiento más equilibrado y estable para este problema de detección de fraude.

En aplicaciones reales, un modelo con alta precisión puede reducir costos operativos asociados a falsas alarmas bancarias.

Figura 9. Comparación entre modelos

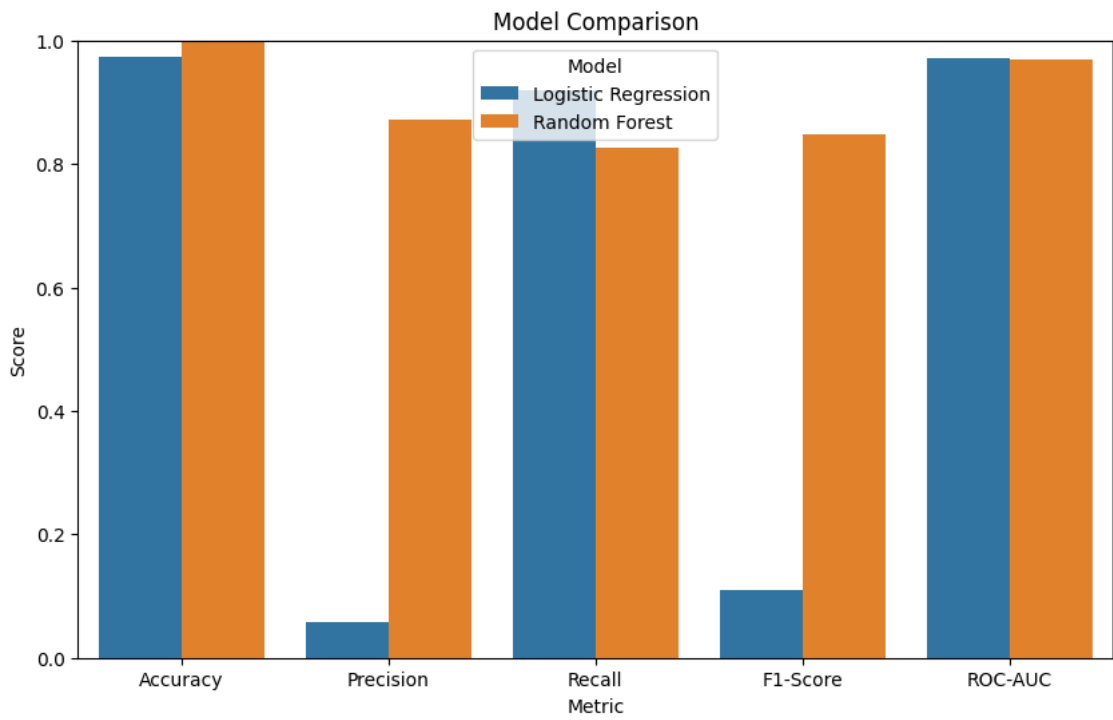
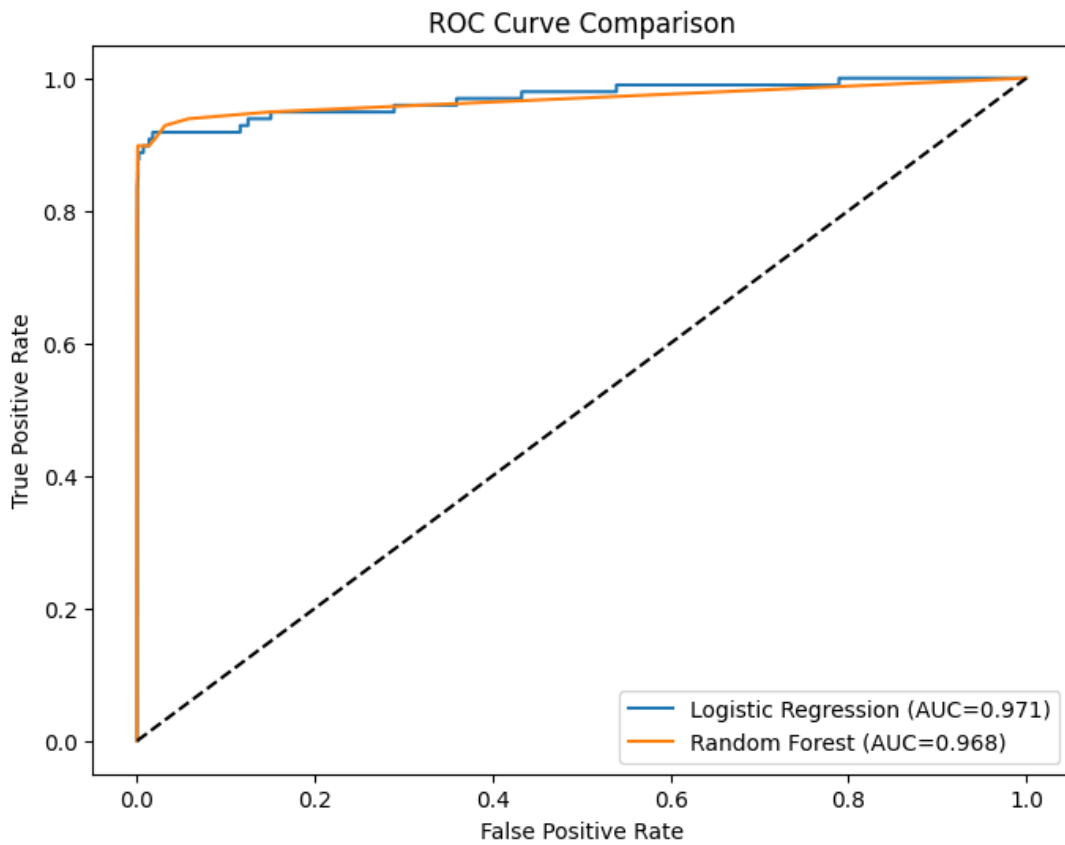


Figura 10. Comparación de ROC Curve



11. Ajuste de Hiperparámetros

11.1 Definición de parámetros

Primero se importó GridSearchCV de la librería de sklearn. El GridSearchCV permitió evaluar sistemáticamente múltiples combinaciones de hiperparámetros mediante validación cruzada.

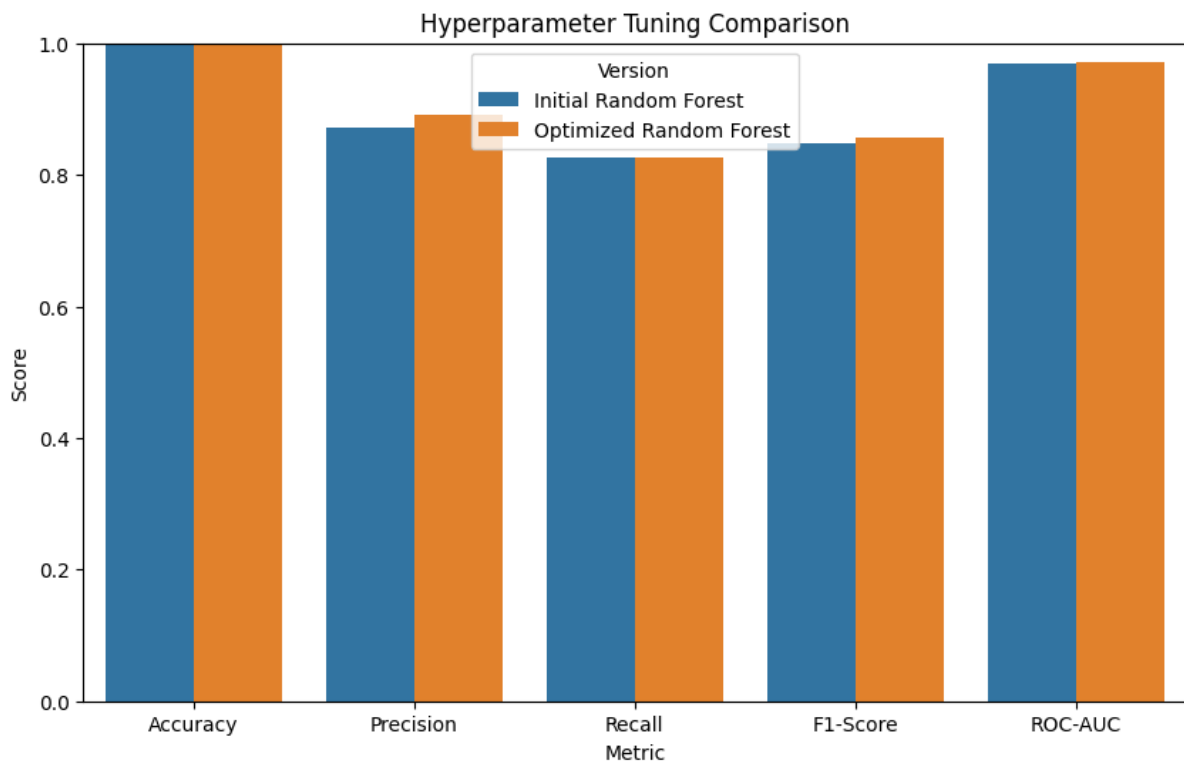
Uso de parámetros como n_estimators (número de árboles), max_depth (profundidad máxima), min_sample_split (mínimo para dividir nodos) y min_sample_leaf

```
param_grid = {  
    'n_estimators': [100, 200],  
    'max_depth': [10, None],  
    'min_samples_split': [2,4],  
    'min_samples_leaf': [1,2]  
}  
scoring='f1',  
cv=2,
```

El parámetro f1-score balancea precisión y el recall, por último, el cv = 2, el cross-validation divide el entrenamiento en 2 partes

	Version	Accuracy	Precision	Recall	F1-Score	ROC-AUC
0	Initial Random Forest	99.95%	87.10%	82.65%	84.82%	96.85%
1	Optimized Random Forest	99.95%	89.01%	82.65%	85.71%	97.08%

Figura 11. Comparación entre RF inicial y RF optimizado



Se usó GridSearchCV para optimizar los hiperparámetros del modelo *Random Forest*. El proceso permitió evaluar múltiples combinaciones (n=32) de parámetros utilizando validación cruzada y F1-score como métrica objetivo. El modelo optimizado mostró una mejora en el equilibrio entre precisión y recall respecto a la versión inicial.

La optimización mejoró principalmente la precisión del modelo, reduciendo falsos positivos sin afectar significativamente el recall.

12. Conclusiones

- El presente proyecto permitió desarrollar un sistema de detección de fraude utilizando técnicas clásicas de ML sobre un dataset compuesto por 284,807 transacciones bancarias y 31 variables. Durante el análisis inicial de datos se identificó un gran desbalance entre las dos clases, donde únicamente 492 transacciones (0.17%) correspondían a fraudes, mientras que 284,315 transacciones fueron clasificadas como legítimas.
- El modelo *Logistic Regression* obtuvo un *accuracy* de 97.55% y un *recall* de 91.84%, logrando detectar 90 de los 98 fraudes presentes en el conjunto de prueba. Sin embargo, presentó una precisión baja de 6.09%, generando 1,389 falsos positivos. Esto demuestra que, aunque el modelo fue muy eficiente detectando fraudes, también clasificó incorrectamente muchas transacciones legítimas como fraudulentas.
- Por otro lado, el modelo *Random Forest* mostró un comportamiento más equilibrado, alcanzando un *accuracy* de 99.95% y una precisión de 96.05%, reduciendo significativamente los falsos positivos. Aunque su *recall* fue menor (74.49%), el modelo consiguió un desempeño general más estable y confiable para aplicaciones reales de detección de fraude.
- El ajuste de hiperparámetros mediante GridSearchCV permitió optimizar el modelo Random Forest evaluando diferentes configuraciones de *n_estimators*, *max_depth* y *min_samples_split*. Después del proceso de optimización, la precisión mejoró de 87.10% a 89.10%, *F1-score* mejoró de 84.82% a 85.71% y *ROC-AUC* mejoró de 96.85% a 97.08%, además de reducir el número de falsos positivos durante la clasificación. Los resultados obtenidos demuestran el potencial de las técnicas clásicas de ML como herramientas de apoyo para sistemas financieros inteligentes orientados a la detección temprana de fraude.

Referencias bibliográficas

1. Machine Learning Group – ULB. (2016). Credit card fraud detection dataset [Dataset]. Kaggle. <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
2. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
3. Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
4. Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied logistic regression* (3rd ed.). Wiley.
5. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
6. Streamlit Inc. (2024). Streamlit documentation. <https://streamlit.io>
7. Python Software Foundation. (2024). Python language reference, version 3.12. <https://www.python.org>
8. McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61.
9. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.
10. Waskom, M. L. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), 3021.